DATA FLOW CFD: 90%

DIAGNOSIS

← NEW ANALYSIS

OBSERVED SYMPTOM

COPY

VS Code does not recognize `cmd + shift + ]` keybindings, while `cmd + shift + [` works. The troubleshooting logs show `shift + cmd + ]` being interpreted as `shift + 9`.

REPRODUCTION PATH

1. Use macOS with a EurKEY keyboard layout.
2. Attempt to bind a command to `cmd + shift + ]`.
3. Press `cmd + shift + ]`.
4. Observe that VS Code does not recognize the keybinding, and the Keybinding Troubleshooting output shows the key as `shift+9` instead of `shift+meta+[` or similar for the physical `]` key.

CONFIDENCE EVIDENCE

COPY

VERIFIED:

- vscode/src/vs/base/browser/keyboardEvent.ts L52: `return EVENT_KEY_CODE_MAP[keyCode] || KeyCode.Unknown;` This line is the fallback that uses a QWERTY-centric map, failing for the EurKEY ']' key.
- User Log: `Received keydown event - modifiers: [shift], code: Digit9, keyCode: 57, key: (` for `cmd + shift + ]`. This shows the browser reports `keyCode 57` for the `]` key when `shift` is pressed.
- User Log: `Converted keydown event - modifiers: [shift], code: Digit9, keyCode: 30 ('9')`. This confirms that `extractKeyCode` (via `EVENT_KEY_CODE_MAP`) maps `keyCode 57` to `KeyCode.KEY_9` (internal value 30).
- User Log: For `cmd + shift + [`: `Converted keydown event - modifiers: [shift,meta], code: BracketLeft, keyCode: 92 ('[')`. This demonstrates that `extractKeyCode` *correctly* maps `keyCode 219` to `KeyCode.US_OPEN_SQUARE_BRACKET` (internal value 92), highlighting the asymmetry and specific nature of the bug for `]`.

CONCEPT TO LEARN

COPY

PLATFORM & LOCALIZATION SPECIFIC QUIRKS

Browser Keyboard Event Normalization Across Layouts

Modern applications must run globally on diverse user setups. Ignoring platform-specific or keyboard layout-specific quirks in event handling can lead to frustrating and inconsistent user experiences, especially for core functionalities like keyboard shortcuts. Proper normalization ensures that a user's intent (e.g., pressing the ']' key) is correctly translated by the application, regardless of how the underlying operating system or browser chooses to report that input.

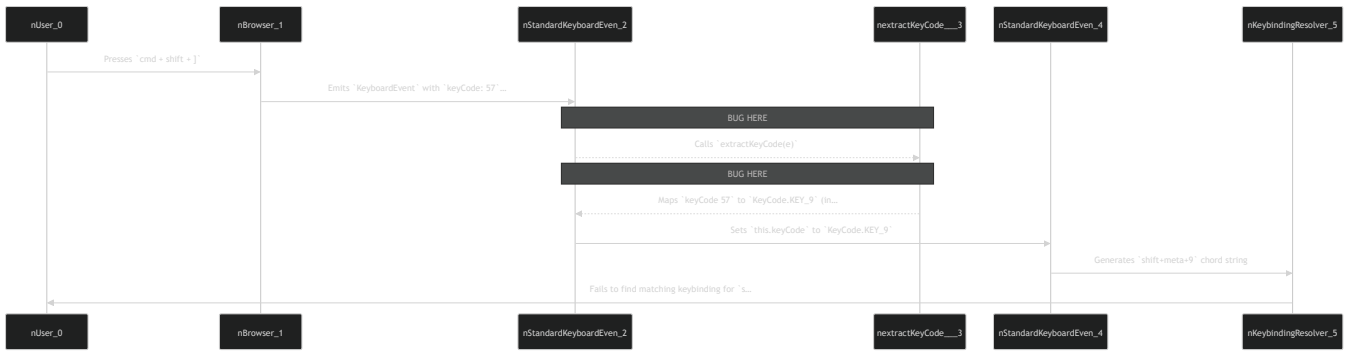
> PATTERN TO AVOID:
> Assuming `e.keyCode` (or `e.code` / `e.key`) is universally consistent for key identification across all operating systems, browsers, and keyboard layouts without implementing explicit normalization logic or quirk-handling for known variations.

💡 *Imagine you're trying to figure out which vegetable is being sold at a market stall. On your local street (standard QWERTY), a red, round vegetable is always a 'tomato'. But if you go to a market in a different city (EurKEY layout), and suddenly their 'tomato' vendor is selling green, leafy vegetables, you'd get confused! You need a special mental note or a small dictionary for that new city to correctly identify the 'tomato' even if it looks different or is called something else locally, or if the vendor points to a different stall (wrong `e.code`/`e.key`) when you ask for 'tomato' (`]`). VS Code needed that special dictionary entry for EurKEY to correctly identify the physical ']' key.*

REAL WORLD ANALOGY

COPY

"Imagine you're trying to figure out which vegetable is being sold at a market stall. On your local street (standard QWERTY), a red, round vegetable is always a 'tomato'. But if you go to a market in a different city (EurKEY layout), and suddenly their 'tomato' vendor is selling green, leafy vegetables, you'd get confused! You need a special mental note or a small dictionary for that new city to correctly identify the 'tomato' even if it looks different or is called something else locally, or if the vendor points to a different stall (wrong `e.code`/`e.key`) when you ask for 'tomato' (`]`). VS Code needed that special dictionary entry for EurKEY to correctly identify the physical ']' key."



EXECUTION SEQUENCE — 🦠 MARKS WHERE THE BUG MANIFESTS

INVARIANT VIOLATIONS

COPY

- The `extractKeyCode` function must always return the `KeyCode` that correctly represents the *intended physical key* (e.g., ']') for keybinding resolution, even when browser `e.keyCode` values vary across platforms/layouts or change with modifier keys.
- The `StandardKeyboardEvent.keyCode` property, used for keybinding resolution, must accurately reflect the base non-modifier key pressed, normalized to a consistent internal `KeyCode` representation.
- Keybinding resolution logic in `KeybindingResolver` relies on `StandardKeyboardEvent.keyCode` accurately representing the user's key press.

TECHNICAL ROOT CAUSE

COPY

`extractKeyCode` (L15-L52) in `vscode/src/vs/base/browser/keyboardEvent.ts` does not correctly handle the `KeyboardEvent.keyCode` value `57` when the physical `]` key is pressed with the `shift` modifier on macOS with a EurKEY layout. This specific browser/OS quirk causes `e.keyCode` to become `57` (which is `9` on QWERTY) instead of a standard `]` `keyCode` (like `221`). As a result, `extractKeyCode` incorrectly maps `keyCode 57` to `KeyCode.KEY_9` (internal 30) via `EVENT_KEY_CODE_MAP`, leading to incorrect keybinding resolution for the intended `]` key.

```
Location: vscode/src/vs/base/browser/keyboardEvent.ts L52
```

CRASH SITE

COPY

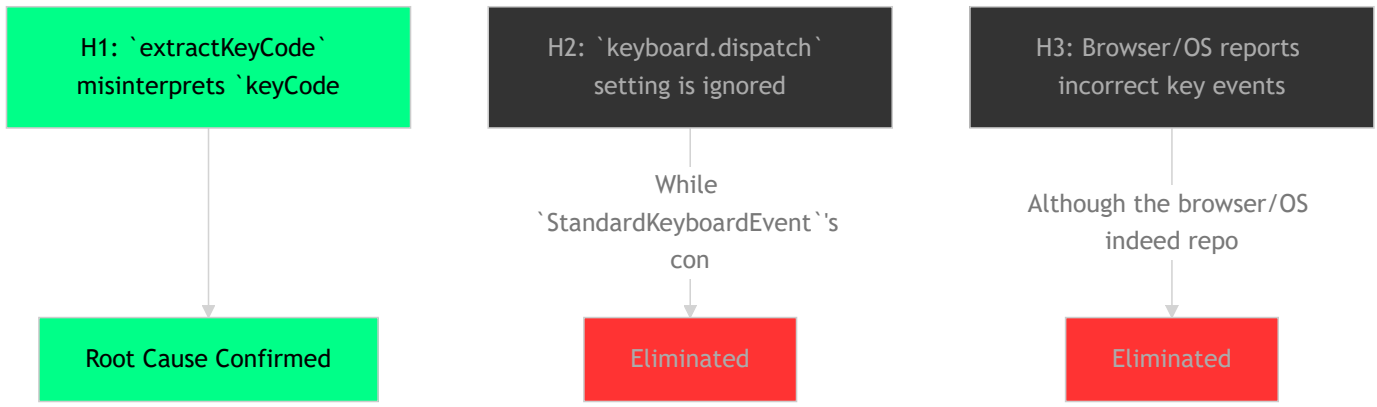
WHERE THE FAILURE BECAME VISIBLE ↓

KeybindingResolver.resolve() L368 — `NoMatchingKb` is returned because `shift+meta+9` does not match the expected keybinding for `shift+meta+]`.

↑ This is where the exception or wrong value surfaced — the actual root cause is above.

ALTERNATIVE HYPOTHESES

- H1: `extractKeyCode` fails to correctly normalize `e.keyCode=57` for the physical `]` key on macOS with EurKEY layout (when `shift` is pressed), causing `StandardKeyboardEvent` to incorrectly store `KeyCode.KEY_9`.
- H2: The `keyboard.dispatch` setting is ignored by `StandardKeyboardEvent`'s constructor, which always uses `e.keyCode` via `extractKeyCode` for keybinding resolution, even when `e.code` should be preferred.
- H3: The browser/OS itself is reporting an incorrect `e.code` (`Digit9`) and `e.keyCode` (`57`) for the physical `]` key on EurKEY, making it inherently impossible for VS Code to correctly identify the key's physical location.



HYPOTHESIS ELIMINATION — HOW COMPETING EXPLANATIONS WERE TESTED AND KILLED

DETERMINISTIC AI FIX PROMPT

COPY PROMPT

Paste this directly into Cursor / Bolt / Copilot to apply the fix:

```
The `extractKeyCode` function in `vscode/src/vs/base/browser/keyboardEvent.ts` needs a specific quirk-handling for macOS with EurKEY layout. When the physical ']' key is pressed with the 'shift' modifier, the browser reports `e.keyCode: 57`, `e.code: Digit9`, and `e.key: '('`. Currently, `extractKeyCode` maps `keyCode 57` to `KeyCode.KEY_9`. Add a condition to `extractKeyCode` to detect this specific combination (macOS, shift key, keyCode 57, code Digit9, key '(') and explicitly return `KeyCode.US_CLOSE_SQUARE_BRACKET` (internal value 93) instead of `KeyCode.KEY_9` (internal value 30). This will correct the key interpretation for the ']' keybinding.
```

MINIMAL CODE FIX

COPY

FILE: VSCODE/SRC/VS/BASE/BROWSER/KEYBOARDEVENT.TS L52

```typescript
// In file: vscode/src/vs/base/browser/keyboardEvent.ts

 function extractKeyCode(e: KeyboardEvent): KeyCode {
        if (e.charCode) {
                // "keypress" events mostly
                const char = String.fromCharCode(e.charCode).toUpperCase();
                return KeyCodeUtils.fromString(char);
        }

        const keyCode = e.keyCode;

        // --- START OF ADDED CODE ---
        // Specific quirk for macOS + EurKEY where physical ']' reports keyCode 57
(QWERTY '9') when Shift is pressed.
        // The browser event reports: `keyCode: 57`, `code: Digit9`, `key: '('` for
physical ']' with Shift.
        // This override ensures the physical ']' key is correctly mapped to
`KeyCode.US_CLOSE_SQUARE_BRACKET`.
        if (platform.isMacintosh && e.shiftKey && keyCode === 57 && e.code ===
'Digit9' && e.key === '(') {
                return KeyCode.US_CLOSE_SQUARE_BRACKET; // Internal KeyCode for ']'
        }
        // --- END OF ADDED CODE ---

        // browser quirks
        if (keyCode === 3) {
                return KeyCode.PauseBreak;
        } else if (browser.isFirefox) {
                switch (keyCode) {
                        case 59: return KeyCode.Semicolon;
                        case 60:
                                if (platform.isLinux) { return KeyCode.IntlBackslash;
}
                                break;
                        case 61: return KeyCode.Equal;
                        // based on: https://developer.mozilla.org/en-
US/docs/Web/API/KeyboardEvent/keyCode#numpad_keys
                        case 107: return KeyCode.NumpadAdd;
                        case 109: return KeyCode.NumpadSubtract;
                        case 173: return KeyCode.Minus;
                        case 224:
                                if (platform.isMacintosh) { return KeyCode.Meta; }
                                break;
                }
        } else if (browser.isWebKit) {
                if (platform.isMacintosh && keyCode === 93) {
                        // the two meta keys in the Mac have different key codes (91
and 93)
                        return KeyCode.Meta;
                } else if (!platform.isMacintosh && keyCode === 92) {
                        return KeyCode.Meta;
```

```
                }
        }

        // cross browser keycodes:
        return EVENT_KEY_CODE_MAP[keyCode] || KeyCode.Unknown;
 }
```

WHY THIS WORKS

The fix directly addresses the misinterpretation of the `KeyboardEvent`. By adding a specific condition to `extractKeyCode`, it overrides the generic `EVENT_KEY_CODE_MAP` lookup for the precise scenario where macOS with a EurKEY layout reports `keyCode 57` (along with `shiftKey`, `e.code: Digit9`, and `e.key: '('`) for the physical `]` key. This ensures that `KeyCode.US_CLOSE_SQUARE_BRACKET` is returned, allowing `StandardKeyboardEvent` to store the correct key, which then enables the `KeybindingResolver` to successfully match `cmd + shift + ]`.

ACTION CENTER

CREATE GITHUB ISSUE

COPY FIX CLI

Create Issue opens a pre-filled GitHub issue in a new tab. You review before submitting.

NEW ANALYSIS